Instituto Federal de Santa Catarina
Câmpus Chapecó
Engenharia de Controle e Automação
Módulo II - Programação I
Professora: *Lara Popov Zambiasi Bazzi Oberderfer*

Introdução à Programação Estruturada Utilizando a Linguagem C

Linguagem C

- Criada por Dennis M. Ritchie e Ken Thompson no laboratório Bell em 1972.
- Baseada na linguagem B (que por sua vez é baseada na linguagem BCPL - *Basic Combined Programming Language* - Linguagem de Programação Básica Combinada*)*.
- Utilizada para criação de programas de diversos tipos (sistemas operacionais, planilhas eletrônicas, processadores de texto,gerenciadores de banco de dados, sistemas de transmissão de dados e telefonia, processadores gráficos, programas para engenharia, etc.

Linguagem C

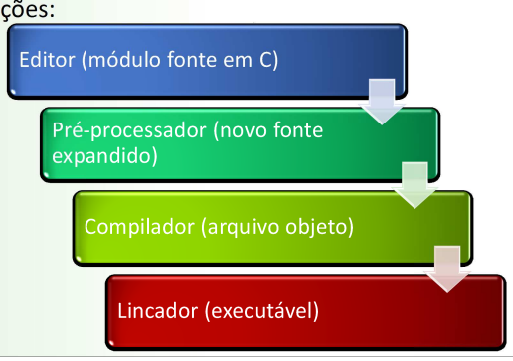
- Programas em C são escritos em um editor de texto e salvos como arquivos em extensão **.c**
  - Os arquivos .c também são conhecidos como código-fonte do programa C.
- Para transformar o código-fonte em um código executável (programa) é necessário utilizar um compilador.
- O **compilador** utilizado na disciplina será o mingw (gcc para windows) presente no editor Dev C++ (distribuição gratuita).

Linguagem C

- Algumas características:
  - C é ***case sensitive***, ou seja, letras maiúsculas e minúsculas fazem TODA diferença.
  - A linguagem é estruturada.
  - Encoraja bons hábitos de programação.
  - Um programa em C pode ser desenvolvido em partes separadas (até mesmo por pessoas distintas) que serão unidas num produto final.
    - Programas feitos em C são rápidos para executar.
    - C é portátil, ou seja, pode ser utilizada para escrever programas para diversos sistemas operacionais ou plataformas.

Linguagem C

- A geração do programa executável a partir do programa fonte obedece a uma sequencia de operações:



Linguagem C

- C linguagem de alto/médio nível
  - C permite a manipulação de bits, bytes e endereço
- 32 palavras-chaves
  - Sintaxe formal do C
- Bibliotecas Padrão C ANSI: especifica padrão de funções que realizam as tarefas mais básicas:

Linguagem C

```
#include <stdio.h>
#include <stdlib.h>
```

- **studio.h**: contém funções, macros e constantes padrões ao compilador.
  Uso em C de quaisquer funções para entrada ou saída de dados
- **stdlib.h**: biblioteca de propósito geral padrão da linguagem de programação C. Ela possui funções envolvendo alocação de memória, controle de processos, conversões e outras.

Diretiva #include

- Utilizada para incluir uma arquivo contendo a definição de funções em linguagem C.
- Estes arquivos, de extensão, .h são denominados bibliotecas de funções.
- Algumas bibliotecas da linguagem C:
  - stdio.h
  - stdlib.h
  - math.h (biblioteca de funções matemáticas)

Programa em C

- Um programa em C é composto por um conjunto de Funções.
  - A função pela qual o programa começa a ser executado chama-se main.
- Após cada comando em C deve-se colocar **;** (ponto-e-vírgula (;)).
- Um programa em C deve ser ***identado*** para que possa ser lido com mais facilidade.

Função main()

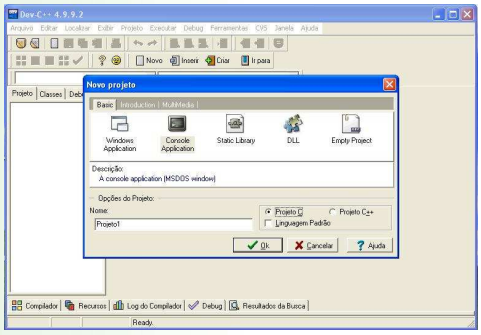
- TODO programa em linguagem C deve possuir uma, e apenas uma, função main().

  - A função main() é o ponto de partida do programa.
  - O código da função main() é delimitado por chaves.
  - Ao final da função main() é utilizado o seguinte código: return 0;

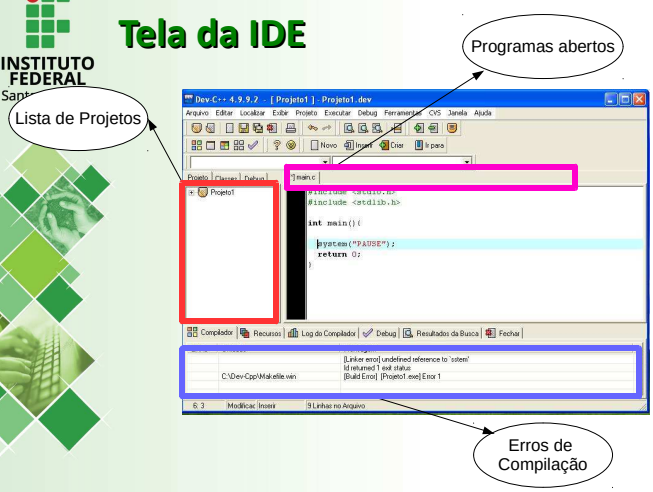
- Sintaxe:

```
int main(void){
  return 0;
}
```


Criando um Novo Projeto



Tela da IDE



Compilando programas no console

- Criar um programa
  - $gedit programa.c
- Compilar o programa
  - $gcc -o programa programa.c
- Executar o programa
  - $./programa

Estrutura básica de um programa

```
tipo nomeFunc(declaração dos parâmetros){
    declaração de variáveis;
    instrução1;
    instrução2;
    …
    instruçãoN;
    return varTipo();
}
```


Variáveis

- Uma variável em C é um espaço de memória reservado para armazenar um certo tipo de dado e tendo um nome para referenciar o seu conteúdo.
  - São associadas ao tipo de dado representado
  - O tipo de dado informa a quantidade de memória que ela ocupará e o modo como um valor deverá ser armazenado e interpretado.
- São identificadas por um nome
  - Escolha nomes significativos
  - 1º caractere deve ser obrigatoriamente uma **letra** ou **_**

Tipos

- %d - int (inteiro – 4 byte)
- %f - float (real – 4 bytes)
- double (real com dupla precisão – 8 bytes)
- %c ou %s - char (caractere – 1 byte)
- void (tipo vazio – 0 byte)
- Exemplo:

```
int idade;
char letra = 'a';
float pi = 3.1416;
```


Exemplo

```
int main(void){
    char c;
    unsigned char uc;
    int i;
    unsigned int ui;
    float f;
    double d;
    printf("\nchar %d",sizeof(c));
    printf("\nunsigned char %d",sizeof(uc));
    printf("\nint %d",sizeof(i));
    printf("\nunsigned int %d",sizeof(ui));
    printf("\nfloat %d",sizeof(f));
    printf("\ndouble %d",sizeof(d));
    return 0;
}
```


Comentários

```
/*
 * Comentários de várias linhas
*/
```

- Pode ser escrito em várias linhas ou uma única linha (até mesmo em uma linha que contém código C).
- A maioria dos copiladores C também aceita o comentário de linha de C++:

// comentário do texto até o final da linha

Exemplos

```
/*
  Este é um comentário
  válido em um programa C
*/
/* Este é um comentário /* inválido
*/ em um programa C */
// Este é um comentário de linha da
// linguagem C++, mas a maioria dos
// compiladores C também o aceitam.
```


Operador de Atribuição

- O **operador de atribuição** em C é o sinal de igual "=". Ao contrário de outras linguagens, o operador de atribuição pode ser utilizado em expressões que também envolvem outros operadores.
- Ex.:

  x = 10;
  y = 4;

  nome = “Lara”;

Operadores Aritméticos

- + (soma)
- - (subtração)
- * (multiplicação)
- / (divisão)
- % (módulo: resto de uma divisão inteira)

Exemplo

```
i+=2;    -> i=i+2;
x*=y+1;  -> x=x*(y+1);
d-=3;    -> d=d-3;
int main(){
    int x,y; x=10; y=3;
    printf("%d\n",x/y);
    printf("%d\n",x%y);
    return 0;
}
```


Operadores

relacionais	*lógicos*
> maior que	&& and
>= maior ou igual	|| ou
< menor	! not
<= menor ou igual	
== igual	
!= não igual	

Exemplo

```
int main(){
    int x, y;
    printf("Digite dois números: ");
    scanf("%d%d",&x,&y);
    printf("%d == %d é %d\n",x,y,x==y);
    printf("%d != %d é %d\n",x,y,x!=y);
    printf("%d <= %d é %d\n",x,y,x<=y);
    printf("%d >= %d é %d\n",x,y,x>=y);
    printf("%d < %d é %d\n",x,y,x< y);
    printf("%d > %d é %d\n",x,y,x> y);
}
```

Incremento e Decremento

- O C fornece operadores diferentes para incrementar variáveis. O operador soma 1 ao seu operando, e o decremento subtrai 1. O aspecto não usual desta notação é que podem ser usado como operadores pré-fixo(++x) ou pós-fixo(x++).
- ++x incrementa x antes de utilizar o seu valor.
- x++ incrementa x depois de ser utilizado.

Exemplo

```
main(){
    int x=0;
    printf("x= %d\n",x++);
    printf("x= %d\n",x);
    printf("x= %d\n",++x);
    printf("x= %d\n",x);
}
```


Precedência

- O nível de precedência é avaliado da esquerda para a direita.
- Os parênteses podem ser usados para alterar a ordem da avaliação.

++

--

*** / %**

+ -

Operador sizeof

- Retorna o tamanho em bytes da variável, ou seja, do tipo que está em seu operando.
- É utilizado para assegurar a portabilidade do programa.

Mãos à obra...

- Exercícios.

Referências Bibliográficas

Introdução à Linguagem C. Versão 2.0. Centro de Computação. UNICAMP. Disponível em: http://www.fsc.ufsc.br/~canzian/root/tutorial-c-unicamp.pdf. Acesso em: 22/03/2013.